

Comparativa de Costos de Complejidad — TDAs

TDA PILA — LIFO (Last In, First Out)			
Método	Big-O Estático	Big-O Dinámico	Notas / Diferencia clave
InicializarPila()	O(1)	O(1)	Estática asigna datos[0] = 0; dinámica asigna tope = null.
Apilar(x)	O(1)	O(1)	Estática escribe en índice del arreglo existente; dinámica crea un nodo nuevo en memoria heap.
Desapilar()	O(1)	O(1)	Estática decreuenta datos[0]; dinámica avanza el puntero tope al siguiente nodo.
Tope()	O(1)	O(1)	Estática accede por datos[datos[0]] (doble índice); dinámica lee tope.dato (un acceso a atributo).
PilaVacía()	O(1)	O(1)	Estática compara datos[0] == 0; dinámica compara tope == null.
PilaLlena()	O(1)	N/A	Solo aplica en estática (comparación contra MAX).

TDA COLA — FIFO (First In, First Out)			
Método	Big-O Estático	Big-O Dinámico	Notas / Diferencia clave
InicializarCola()	O(1)	O(1)	Estática asigna datos[0] = 0; dinámica asigna frente = fondo = null.
Acolar(x)	O(1)	O(1)	Estática escribe en índice del arreglo; dinámica crea nodo y actualiza puntero fondo.
Desacolar()	O(n)	O(1)	Estática O(n) por desplazamiento de elementos; dinámica O(1) solo actualiza puntero frente.
Primero()	O(1)	O(1)	Estática accede a datos[1]; dinámica lee frente.dato.
ColaVacía()	O(1)	O(1)	Estática compara datos[0] == 0; dinámica compara frente == null.
ColaLlena()	O(1)	N/A	Solo estática O(1), compara datos[0] == MAX; dinámica no existe, sin límite de tamaño.

TDA COLA CON PRIORIDAD			
Método	Big-O Estático	Big-O Dinámico	Notas / Diferencia clave
InicializarColaPrioridad()	O(1)	O(1)	Reserva memoria para el arreglo de subcolas o estructura inicial.
AcolarPrioridad(x, p)	O(n)	O(n)	Estática: Inserta y desplaza para mantener orden. Dinámica: Busca la subcola "p" e inserta al final (es O(n) si debe buscar la posición en una lista única).
Desacolar()	O(n)	O(1)	Estática: Debe buscar y eliminar el de mayor prioridad (y desplazar). Dinámica: Extrae directamente de la primera subcola no vacía.
Primero()	O(1)	O(1)	Acceso directo al primer elemento de la subcola con mayor prioridad.
Prioridad()	O(1)	O(1)	O(1) Devuelve el valor de prioridad del elemento al frente de la estructura.
ColaVacía()	O(1)	O(1)	Verifica si la cantidad total de elementos es cero.

TDA CONJUNTO			
Método	Big-O Estático	Big-O Dinámico	Notas / Diferencia clave
InicializarConjunto()	O(1)	O(1)	Inicializa estructura vacía.
Agregar(x)	O(n)	O(n)	Ambas verifican duplicados recorriendo todos los elementos antes de insertar.
Sacar(x)	O(n)	O(n)	Busca x linealmente. Estática: reemplaza por último. Dinámica: reenlaza nodos.
Pertenece(x)	O(n)	O(n)	Búsqueda lineal en ambas (sin orden garantizado).
Elegir()	O(1)	O(1)	Retorna primer elemento directamente.
ConjuntoVacio()	O(1)	O(1)	Compara contador/puntero.

TDA DICCIONARIO SIMPLE			
Método	Big-O Estático	Big-O Dinámico	Notas / Diferencia clave
InicializarDiccionario()	O(1)	O(1)	Inicializa estructura vacía.
Agregar(clave, valor)	O(n)	O(n)	Busca si la clave existe (lineal) antes de insertar/reemplazar.
Eliminar(clave)	O(n)	O(n)	Busca la clave linealmente para eliminarla.

TDA DICCIONARIO SIMPLE			
Método	Big-O Estático	Big-O Dinámico	Notas / Diferencia clave
Recuperar(clave)	$O(n)$	$O(n)$	Búsqueda lineal por clave.
Claves()	$O(n)$	$O(n)$	Recorre todos los elementos para armar el arreglo de claves.

TDA DICCIONARIO MÚLTIPLE			
Método	Big-O Estático	Big-O Dinámico	Notas / Diferencia clave
InicializarDiccionario()	$O(1)$	$O(1)$	Inicializa estructura vacía.
Agregar(clave, valor)	$O(n)$	$O(n)$	Busca clave ($O(n)$) + verifica duplicado en el conjunto de valores ($O(n)$).
Eliminar(clave)	$O(n)$	$O(n)$	Busca y elimina la clave completa con todos sus valores.
EliminarValor(clave, v)	$O(n)$	$O(n)$	Busca clave ($O(n)$) + busca y elimina valor en el conjunto ($O(n)$).
Recuperar(clave)	$O(n)$	$O(n)$	Búsqueda lineal por clave, retorna conjunto.
Claves()	$O(n)$	$O(n)$	Recorre todos los elementos para armar el arreglo de claves.